

Distributional Reinforcement Learning | Part 2

Jonas Müller

June 22, 2026

Outline

- 1 Recap
- 2 Linear Function Approximation
- 3 Deep Distributional Reinforcement Learning

Quick Recap

As seen, for a finite MDP with policy π and discount $\gamma \in [0, 1)$, we define

$$V^\pi(x) = \mathbb{E}[G^\pi(x)], \quad \eta^\pi(x) = \text{Law}(G^\pi(x)).$$

The corresponding Bellman equations are

$$V^\pi = \mathcal{T}^\pi V^\pi, \quad \eta^\pi = \mathcal{T}^\pi \eta^\pi,$$

As seen, return distributions can be represented categorically or through quantiles and learned using CTD or QTD updates.

Focus of this talk

How do we share parameters across a very large state space, first with fixed linear features and then with learned neural-network features?

Function Approximation and Aliasing

A tabular estimate has a separate parameter for every state. Function approximation shares parameters across states.

- It makes large or continuous state spaces tractable.
- It generalizes predictions to states not yet observed.
- It also creates **aliasing**: one parameter affects several states.

If aliased states require different predictions, no parameter choice is exact. With bootstrapping, their approximation error can propagate to states that are not themselves aliased.

Two distinct approximations

Distributional RL approximates both the output distribution and the mapping from states to those distributions.

Linear Value Function Approximation

A state representation is a map

$$\phi : \mathcal{X} \rightarrow \mathbb{R}^n, \quad \phi(x) = (\phi_1(x), \dots, \phi_n(x))^\top.$$

For weights $w \in \mathbb{R}^n$, the approximation is

$$V_w(x) = \phi(x)^\top w, \quad \nabla_w V_w(x) = \phi(x).$$

For finite $\mathcal{X}$, stack feature rows into $\Phi \in \mathbb{R}^{|\mathcal{X}| \times n}$:

$$V_w = \Phi w, \quad \mathcal{F}_\phi = \{\Phi w : w \in \mathbb{R}^n\}.$$

For action values either use $\phi : \mathcal{X} \times \mathcal{A} \rightarrow \mathbb{R}^n$ and $Q_w(x, a) = \phi(x, a)^\top w$, or shared $\phi(x)$ with per-action weights w_a .

The Best Linear Approximation

Choose a state-weighting distribution $\xi \in \mathcal{P}(\mathcal{X})$ and define

$$\|V - V'\|_{\xi,2} = \left(\sum_{x \in \mathcal{X}} \xi(x) (V(x) - V'(x))^2 \right)^{1/2}.$$

The representation problem is

$$\min_{w \in \mathbb{R}^n} \|V^\pi - V_w\|_{\xi,2}.$$

With $\Xi = \text{diag}(\xi(x) : x \in \mathcal{X})$, full support of ξ , and linearly independent columns of Φ , the unique solution is

$$w^* = (\Phi^\top \Xi \Phi)^{-1} \Phi^\top \Xi V^\pi.$$

The choice of ξ says which states matter most; it is part of the approximation problem, not a harmless implementation detail.

Projection Back into the Feature Space

In general, $V \in \mathcal{F}_\phi$ does not imply $\mathcal{T}^\pi V \in \mathcal{F}_\phi$. Define the weighted orthogonal projection

$$(\Pi_{\phi,\xi} V)(x) = \phi(x)^\top w^*, \quad w^* \in \arg \min_w \|V - V_w\|_{\xi,2}.$$

Approximate dynamic programming therefore iterates

$$V_{k+1} = \Pi_{\phi,\xi} \mathcal{T}^\pi V_k.$$

- $\mathcal{T}^\pi$ performs the Bellman update.
- $\Pi_{\phi,\xi}$ removes components that the features cannot express.
- Repeated projection can compound error (the diffusion effect).

When Is the Projected Operator Stable?

Let ξ_π be a fully supported steady-state distribution under π . Then both P^π and Π_{ϕ, ξ_π} are nonexpansions in the ξ_π -weighted L_2 norm. Hence

$$\|\Pi_{\phi, \xi_\pi} \mathcal{T}^\pi V - \Pi_{\phi, \xi_\pi} \mathcal{T}^\pi V'\|_{\xi_\pi, 2} \leq \gamma \|V - V'\|_{\xi_\pi, 2}.$$

It has a unique fixed point

$$\hat{V}^\pi = \Pi_{\phi, \xi_\pi} \mathcal{T}^\pi \hat{V}^\pi,$$

with error bound

$$\|\hat{V}^\pi - V^\pi\|_{\xi_\pi, 2} \leq \frac{1}{\sqrt{1 - \gamma^2}} \|\Pi_{\phi, \xi_\pi} V^\pi - V^\pi\|_{\xi_\pi, 2}.$$

Monte Carlo as Stochastic Gradient Descent

Given independent state–return samples (x_k, g_k) , use the sample loss

$$L(w) = (g_k - \phi(x_k)^\top w)^2.$$

Absorbing the factor 2 into the step size gives

$$w \leftarrow w + \alpha_k (g_k - \phi(x_k)^\top w) \phi(x_k).$$

This is genuine stochastic gradient descent: the target g_k does not depend on w . Under standard sampling and step-size conditions, it approaches the weighted least-squares solution.

Semi-Gradient Temporal-Difference Learning

For a transition (x_k, a_k, r_k, x'_k) , replace the Monte Carlo return by the bootstrapped target

$$r_k + \gamma V_w(x'_k) = r_k + \gamma \phi(x'_k)^\top w.$$

The update is

$$w \leftarrow w + \alpha_k \underbrace{\left(r_k + \gamma \phi(x'_k)^\top w - \phi(x_k)^\top w \right)}_{\text{TD error}} \phi(x_k).$$

It is a *semi*-gradient: while differentiating, the target is treated as fixed. Differentiating the full squared Bellman error would also produce a term involving $\phi(x'_k)$.

Target Weights Explain the Projection Connection

Freeze a second vector $\tilde{w}$ inside the target:

$$w \leftarrow w + \alpha_k (r_k + \gamma \phi(x'_k)^\top \tilde{w} - \phi(x_k)^\top w) \phi(x_k).$$

This is SGD on a fixed-target loss and approximately computes one projected Bellman step:

$$\Phi w^* = \Pi_{\phi, \xi} \mathcal{T}^\pi V_{\tilde{w}}.$$

Setting $\tilde{w} = w$ recovers ordinary semi-gradient TD, but also creates a feedback loop. Function approximation, bootstrapping, and an unsuitable sampling distribution can then produce divergence (Baird's counterexample).

Why “Linear Distribution” Needs Care

Probability distributions do not form a vector space:

$$\eta_{w_1+w_2}(x) \neq \eta_{w_1}(x) + \eta_{w_2}(x)$$

because the right side has total mass 2.

Book's convention

“Linear” distributional approximation means that the *parameters of a probability representation* depend linearly on features.

There are therefore two approximation layers:

- 1 a finite probability representation (categorical or quantile),
- 2 feature aliasing across states.

Linear Quantile TD: Representation and Loss

With m quantile locations and weights $w_1, \dots, w_m \in \mathbb{R}^n$,

$$\eta_w(x) = \frac{1}{m} \sum_{i=1}^m \delta_{\phi(x)^\top w_i}, \quad \tau_i = \frac{2i-1}{2m}.$$

Define the quantile loss

$$\rho_\tau(\Delta) = |\mathbb{1}_{\{\Delta < 0\}} - \tau| |\Delta|.$$

For one target z , head i uses

$$L_{\tau_i}(w_i) = \rho_{\tau_i}(z - \phi(x)^\top w_i),$$

whose subgradient is proportional to $-(\tau_i - \mathbb{1}_{\{z < \phi(x)^\top w_i\}})\phi(x)$.

Linear QTD: Distributional Semi-Gradient

A transition (x, a, r, x') supplies m target samples

$$g_j = r + \gamma \phi(x')^\top w_j, \quad j = 1, \dots, m.$$

Averaging the quantile subgradient over these targets yields

$$w_i \leftarrow w_i + \alpha \frac{1}{m} \sum_{j=1}^m \underbrace{(\tau_i - \mathbb{1}_{\{g_j < \phi(x)^\top w_i\}})}_{\text{quantile TD error}} \phi(x).$$

As in scalar TD, g_j is treated as constant during differentiation. Unlike the scalar TD error, each averaged quantile error is bounded.

Linear Categorical TD: Prediction

Fix support locations $\theta_1 < \dots < \theta_m$. Linear logits are transformed into probabilities by a softmax:

$$p_i(x; w) = \frac{\exp(\phi(x)^\top w_i)}{\sum_{j=1}^m \exp(\phi(x)^\top w_j)}, \quad \eta_w(x) = \sum_{i=1}^m p_i(x; w) \delta_{\theta_i}.$$

The softmax is essential: arbitrary linear coefficients need not be nonnegative or sum to one.

Linear CTD: Target, Loss, and Update

Project the Bellman-updated next distribution back to the fixed support:

$$\bar{\eta}(x) = \Pi_c(b_{r,\gamma})_{\#} \eta_w(x') = \sum_{i=1}^m \bar{p}_i \delta_{\theta_i}.$$

Cross-entropy is the matching loss for softmax probabilities:

$$L(w) = - \sum_{i=1}^m \bar{p}_i \log p_i(x; w).$$

Its semi-gradient update has the same template as linear TD and QTD:

$$w_i \leftarrow w_i + \textit{alpha} \underbrace{(\bar{p}_i - p_i(x; w))}_{\text{CTD error}} \phi(x).$$

Thus all three methods have the form $w_i \leftarrow w_i + \alpha \varepsilon_i \phi(x)$.

From Fixed Features to Learned Features

A deep network replaces the hand-designed map $\phi(x)$ by a learned representation. Its penultimate layer can still be read as

$$\phi_w(x) \in \mathbb{R}^n,$$

followed by linear prediction heads.

- Earlier layers learn which state distinctions matter.
- Final layers map those features to values or distribution parameters.
- Backpropagation updates both simultaneously.

Linear approximation is therefore not discarded; it remains the final layer and the conceptual model for the deep semi-gradient update.

DQN: Prediction, Behavior, and Experience

DQN maps an observation x to one value per action:

$$Q_w(x, a), \quad a \in \mathcal{A}.$$

In the canonical Atari setup:

- four preprocessed 84×84 frames form the network input;
- convolutional layers and a fully connected layer produce 512 features;
- a linear output layer produces $|\mathcal{A}|$ action values;
- an ε -greedy policy acts from Q_w ;
- transitions are stored and resampled from replay memory.

DQN: Semi-Gradient Learning

With online weights w , target weights $\tilde{w}$, and transition (x, a, r, x') , define

$$y = r + \gamma \max_{a' \in \mathcal{A}} Q_{\tilde{w}}(x', a').$$

The squared loss and semi-gradient update are

$$L(w) = (y - Q_w(x, a))^2,$$

$$w \leftarrow w + \alpha(y - Q_w(x, a))\nabla_w Q_w(x, a).$$

If $Q_w(x, a) = \phi(x, a)^\top w$, then $\nabla_w Q_w = \phi$: this is exactly the linear update. Replay reduces temporal correlation; the frozen target network slows the moving-target feedback loop.

Distributional Network Output

Deep distributional RL changes the output from one scalar per action to m distribution parameters per action:

$$x \mapsto \mathbb{R}^{|\mathcal{A}| \times m}.$$

The interpretation determines the algorithm:

- **C51**: fixed locations, learned softmax probabilities;
- **QR-DQN**: equal probabilities, learned quantile locations;
- **IQN**: quantile level τ becomes an additional input.

Behavior is usually risk-neutral: derive $Q_w(x, a)$ from the predicted distribution and do ε -greedy action selection as in DQN.

C51: Prediction and Behavior

For fixed $\theta_1 < \dots < \theta_m$, the network outputs logits $\varphi_i(x, a; w)$ and

$$p_i(x, a; w) = \frac{e^{\varphi_i(x, a; w)}}{\sum_{j=1}^m e^{\varphi_j(x, a; w)}}, \quad \eta_w(x, a) = \sum_{i=1}^m p_i(x, a; w) \delta_{\theta_i}.$$

The induced action value is

$$Q_w(x, a) = \sum_{i=1}^m p_i(x, a; w) \theta_i.$$

Original C51 uses $m = 51$ evenly spaced locations; support choice imposes an important inductive bias because projected targets outside it are clipped.

C51: Bellman Target and Cross-Entropy

Select the target-network greedy action

$$a_{\tilde{w}}(x') \in \arg \max_{a'} Q_{\tilde{w}}(x', a').$$

Construct and project the target

$$\bar{\eta}(x, a) = \Pi_c(b_{r,\gamma})_{\#} \eta_{\tilde{w}}(x', a_{\tilde{w}}(x')) = \sum_{j=1}^m \bar{p}_j \delta_{\theta_j}.$$

Then minimize

$$L(w) = - \sum_{j=1}^m \bar{p}_j \log p_j(x, a; w).$$

This is the nonlinear analogue of linear CTD, except shared early-layer weights influence all particles and cannot be updated independently.

QR-DQN: Quantile Prediction

QR-DQN interprets the m outputs as locations of an equally weighted quantile distribution:

$$\eta_w(x, a) = \frac{1}{m} \sum_{i=1}^m \delta_{\theta_i(x, a; w)}, \quad Q_w(x, a) = \frac{1}{m} \sum_{i=1}^m \theta_i(x, a; w).$$

With $a_{\tilde{w}}(x')$ greedy by this mean, the pairwise residuals are

$$\Delta_{ij} = r + \gamma \theta_j(x', a_{\tilde{w}}(x'); \tilde{w}) - \theta_i(x, a; w).$$

Every predicted quantile is compared with every target quantile.

QR-DQN: Quantile Huber Loss

For $\tau_i = (2i - 1)/(2m)$, QR-DQN minimizes

$$L(w) = \frac{1}{m} \sum_{i,j=1}^m \rho_{\tau_i}^H(\Delta_{ij}), \quad \rho_{\tau}^H(u) = |\mathbb{1}_{\{u < 0\}} - \tau| H_1(u),$$

where

$$H_1(u) = \begin{cases} \frac{1}{2}u^2, & |u| \leq 1, \\ |u| - \frac{1}{2}, & |u| > 1. \end{cases}$$

The quantile asymmetry places different heads at different parts of the distribution; the Huber transition gives smooth small-error gradients and limits the influence of large errors.

IQN: Make the Quantile Level an Input

C51 and QR-DQN output a fixed finite parameter vector. IQN instead models the quantile function itself:

$$(x, a, \tau) \mapsto \theta_w(x, a, \tau) \approx F_{\eta(x,a)}^{-1}(\tau).$$

The level is encoded by

$$\varphi(\tau) = (\cos(\pi \tau i))_{i=0}^{M-1},$$

transformed by a fully connected layer, and combined multiplicatively with the state features. The final action heads satisfy

$$\theta_w(x, a, \tau) = \phi_w(x, \tau)^\top w_a.$$

IQN: Behavior and Learning

Approximate the mean with independent $\tau_i \sim \mathcal{U}(0, 1)$:

$$Q_w(x, a) \approx \frac{1}{m} \sum_{i=1}^m \theta_w(x, a, \tau_i).$$

For independent online levels τ_i and target levels τ'_j , minimize pairwise losses of

$$r + \gamma \theta_{\tilde{w}}(x', a_{\tilde{w}}(x'), \tau'_j) - \theta_w(x, a, \tau_i).$$

Sampling many level pairs reduces gradient variance. IQN approximates the whole quantile function without committing to fixed output locations.

IQN Makes Risk Sensitivity Simple

Risk-neutral behavior samples $\tau \sim \mathcal{U}(0, 1)$. Lower-tail CVaR at level $\bar{\tau}$ is

$$\text{CVaR}_{\bar{\tau}}(G_w(x, a)) = \frac{1}{\bar{\tau}} \int_0^{\bar{\tau}} \theta_w(x, a, \tau) d\tau.$$

Approximate it by changing only the sampling distribution:

$$\text{CVaR}_{\bar{\tau}}(G_w(x, a)) \approx \frac{1}{m} \sum_{i=1}^m \theta_w(x, a, \tau_i), \quad \tau_i \sim \mathcal{U}(0, \bar{\tau}).$$

Thus an implicit representation separates learning the return distribution from choosing how that distribution should guide behavior.

How Distributional Predictions Shape Features

Split the network into learned features and final prediction heads.

- DQN trains the representation to predict one conditional mean per action.
- C51 trains it through m probability errors per action.
- QR-DQN/IQN train it through many pairwise quantile residuals.

The type and number of predictions therefore change the learned state representation. This richer auxiliary prediction problem is a leading explanation for improved control performance even when actions are ultimately selected by the mean.

Takeaways

- ➊ Linear approximation introduces state aliasing and a weighted projection problem; the sampling distribution is central to stability.
- ➋ Semi-gradient TD tracks a projected Bellman process, and target weights make that relationship explicit.
- ➌ Linear QTD and CTD make distribution *parameters* linear in features.
- ➍ Deep methods learn the features and retain the same prediction–target–semi-gradient structure.
- ➎ C51, QR-DQN, and IQN differ mainly in how they parameterize and train the output distribution; that choice also shapes the learned representation.

References

- Bellemare, Dabney, Rowland (2023): *Distributional Reinforcement Learning*, Chapters 9–10.
- Bellemare, Dabney, Munos (2017): A Distributional Perspective on Reinforcement Learning.
- Dabney et al. (2018): Distributional Reinforcement Learning with Quantile Regression.
- Dabney et al. (2018): Implicit Quantile Networks for Distributional Reinforcement Learning.